

FPGA ASSESSMENT 2

Anonymous public copy

Personal and institutional identifiers removed.

Contents

1	Program 1 : Half Adder	3
1.1	Verilog Code	3
1.2	Testbench	3
1.3	Simulation Output	3
2	Program 2 : Full Adder	4
2.1	Verilog Code	4
2.2	Testbench	4
2.3	Simulation Output	5
3	Program 3 : Half Subtractor	6
3.1	Verilog Code	6
3.2	Testbench	6
3.3	Simulation Output	7
4	Program 4 : Full Subtractor	8
4.1	Verilog Code	8
4.2	Testbench	8
4.3	Simulation Output	9
5	Program 5 : 2-to-4 Decoder	10
5.1	Verilog Code	10
5.2	Testbench	10
5.3	Simulation Output	11
6	Program 6 : 4-to-2 Encoder	12
6.1	Verilog Code	12
6.2	Testbench	12
6.3	Simulation Output	13
7	Program 7 : 2-to-1 Multiplexer	14
7.1	Verilog Code	14
7.2	Testbench	14
7.3	Simulation Output	15
8	Program 8 : 4-to-1 Multiplexer	16
8.1	Verilog Code	16
8.2	Testbench	16
8.3	Simulation Output	17
9	Program 9 : 4-bit Ripple Carry Adder	18
9.1	Verilog Code	18
9.2	Testbench	18
9.3	Simulation Output	19

10 Program 10 : 4-bit Adder/Subtractor	20
10.1 Verilog Code	20
10.2 Testbench	20
10.3 Simulation Output	21
11 Program 11 : 4-bit Multiplier	22
11.1 Verilog Code	22
11.2 Testbench	22
11.3 Simulation Output	23
12 Program 12 : Arithmetic Unit (S=0: Y=A-B, S=1: Y=B-A)	24
12.1 Verilog Code	24
12.2 Testbench	24
12.3 Simulation Output	25
13 Program 13 : Arithmetic Circuit ($Y=A^2 + 6A$)	26
13.1 Verilog Code	26
13.2 Testbench	26
13.3 Simulation Output	27
14 Program 14 : FPGA Design Implementation Experiment	28
14.1 Verilog Code	28
14.2 Verilog Code	28
14.3 Verilog Code	28
14.4 Testbench	29
14.5 Pin Assignment	29
14.6 Simulation Output	31
14.7 FPGA Implementation Output	32
14.8 FPGA Implementation Output	35

1 Program 1 : Half Adder

1.1 Verilog Code

```
1 module ha(Sum, Carry, a, b);
2     output Sum, Carry;
3     input a, b;
4     xor (Sum, a, b);
5     and (Carry, a, b);
6
7 endmodule
```

1.2 Testbench

```
1 module t_ha();
2     reg a, b;
3     wire s, c; ha h1(s, c, a, b);
4     initial
5     begin
6         a = 0; b = 0;
7         #5 a = 0; b = 1;
8         #5 a = 1; b = 0;
9         #5 a = 1; b = 1;
10        #10 $stop;
11    end endmodule
```

1.3 Simulation Output

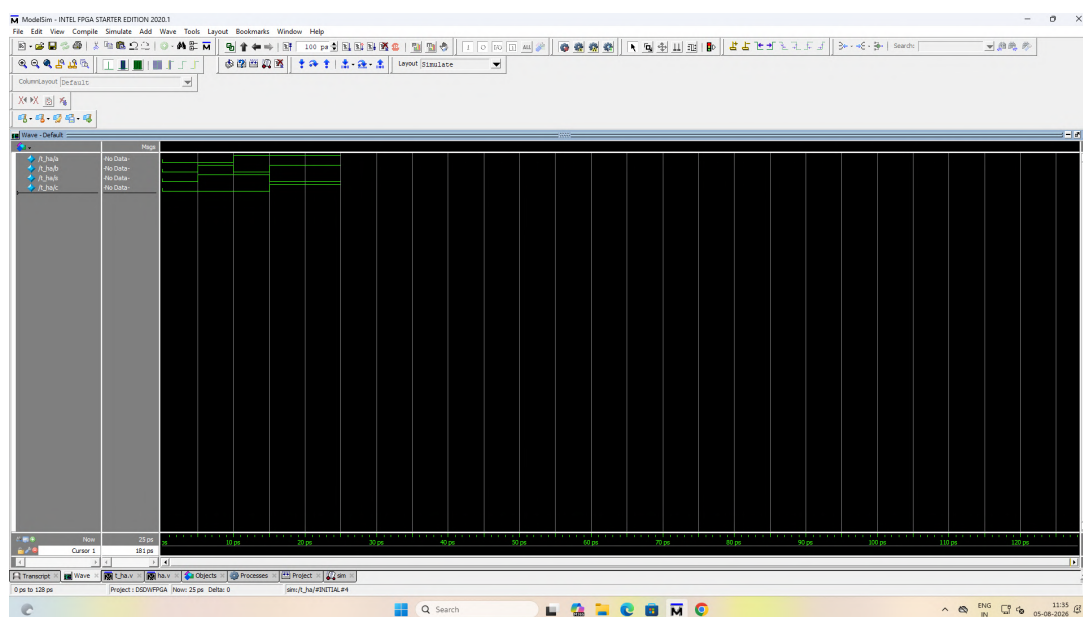


Figure 1: Simulation Output

2 Program 2 : Full Adder

2.1 Verilog Code

```
1 module fa(Sum, Carry, a, b, c);
2     output Sum, Carry;
3     input a, b, c;
4     wire w1, w2, w3;
5
6     ha h1(w1, w2, a, b);
7     ha h2(Sum, w3, w1, c);
8     or (Carry, w2, w3);
9
10 endmodule
```

2.2 Testbench

```
1 module t_fa();
2 reg a, b, cin;
3 wire s, c;
4 fa f1(s, c, a, b, cin);
5 initial
6 begin
7     a = 0; b = 0; cin = 0;
8     #5 a = 0; b = 0; cin = 1;
9     #5 a = 0; b = 1; cin = 0;
10    #5 a = 0; b = 1; cin = 1;
11    #5 a = 1; b = 0; cin = 0;
12    #5 a = 1; b = 0; cin = 1;
13    #5 a = 1; b = 1; cin = 0;
14    #5 a = 1; b = 1; cin = 1;
15    #10 $stop;
16 end
17 endmodule
```

2.3 Simulation Output

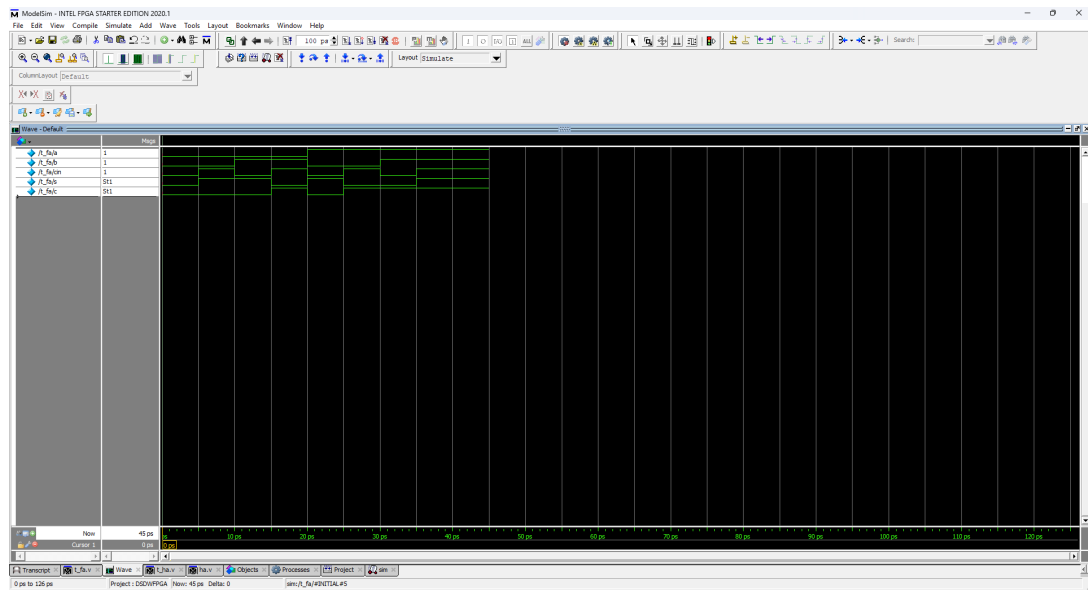


Figure 2: Simulation Output

3 Program 3 : Half Subtractor

3.1 Verilog Code

```
1 module hs(Diff, Borrow, a, b);
2 output Diff, Borrow;
3 input a, b;
4 wire w1;
5 not (w1, a);
6 xor (Diff, a, b);
7 and (Borrow, w1, b);
8 endmodule
```

3.2 Testbench

```
1 module t_hs();
2 reg a, b;
3 wire diff, borrow;
4 hs h1(diff, borrow, a, b);
5 initial
6 begin
7 a = 0; b = 0;
8 #5 a = 0; b = 1;
9 #5 a = 1; b = 0;
10 #5 a = 1; b = 1;
11 #10 $stop;
12 end
13 endmodule
```

3.3 Simulation Output

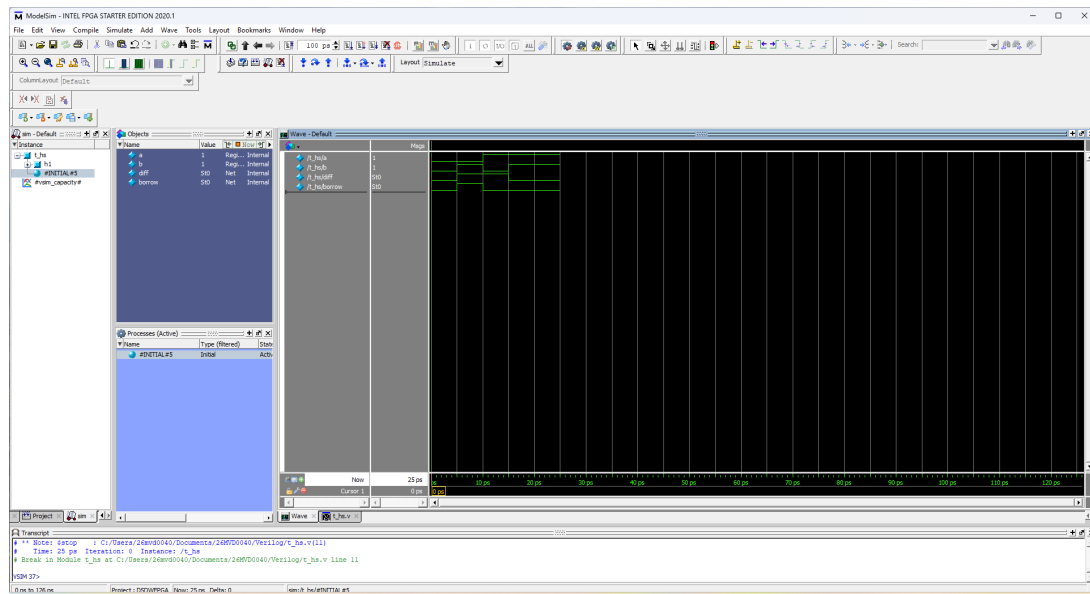


Figure 3: Simulation Output

4 Program 4 : Full Subtractor

4.1 Verilog Code

```
1 module fs(Diff, Borrow, a, b, bin);
2 output Diff, Borrow;
3 input a, b, bin;
4 wire w1, w2, w3;
5 hs h1(w1, w2, a, b);
6 hs h2(Diff, w3, w1, bin);
7 or (Borrow, w2, w3);
8 endmodule
```

4.2 Testbench

```
1 module t_fs();
2 reg a, b, bin;
3 wire diff, borrow;
4 fs f1(diff, borrow, a, b, bin);
5 initial
6 begin
7 a = 0; b = 0; bin = 0;
8 #5 a = 0; b = 0; bin = 1;
9 #5 a = 0; b = 1; bin = 0;
10 #5 a = 0; b = 1; bin = 1;
11 #5 a = 1; b = 0; bin = 0;
12 #5 a = 1; b = 0; bin = 1;
13 #5 a = 1; b = 1; bin = 0;
14 #5 a = 1; b = 1; bin = 1;
15 #10 $stop;
16 end
17 endmodule
```

4.3 Simulation Output

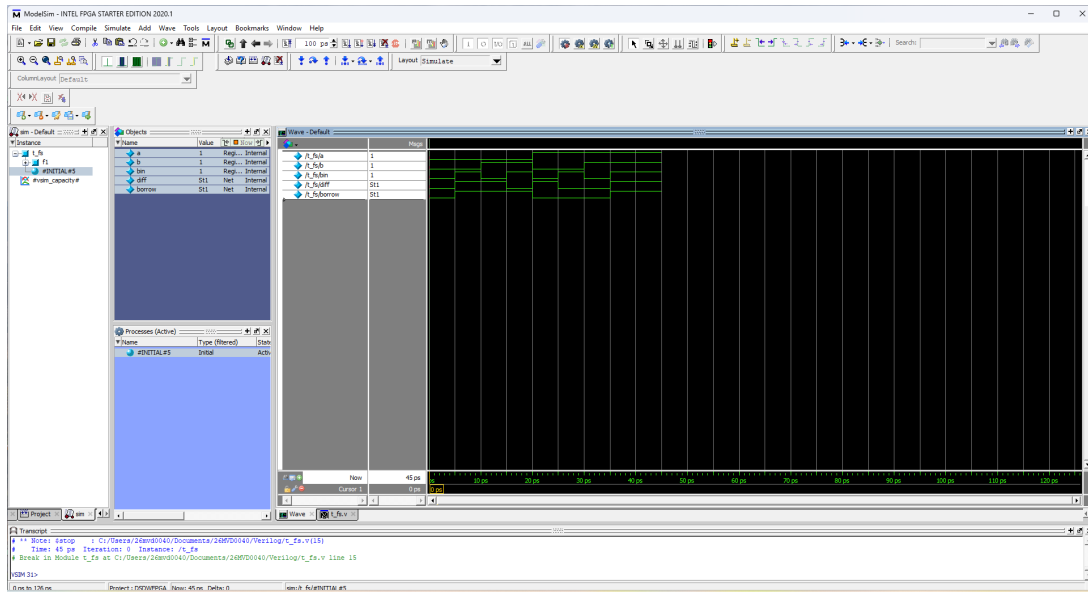


Figure 4: Simulation Output

5 Program 5 : 2-to-4 Decoder

5.1 Verilog Code

```
1 module dec(Y, a, b);
2 output [3:0] Y;
3 input a, b;
4 wire w1, w2;
5 not (w1, a);
6 not (w2, b);
7 and (Y[0], w1, w2);
8 and (Y[1], w1, b);
9 and (Y[2], a, w2);
10 and (Y[3], a, b);
11 endmodule
```

5.2 Testbench

```
1 module t_dec();
2 reg a, b;
3 wire [3:0] y;
4 dec d1(y, a, b);
5 initial begin a = 0; b = 0;
6 #5 a = 0; b = 1;
7 #5 a = 1; b = 0;
8 #5 a = 1; b = 1;
9 #10 $stop;
10 end
11 endmodule
```

5.3 Simulation Output

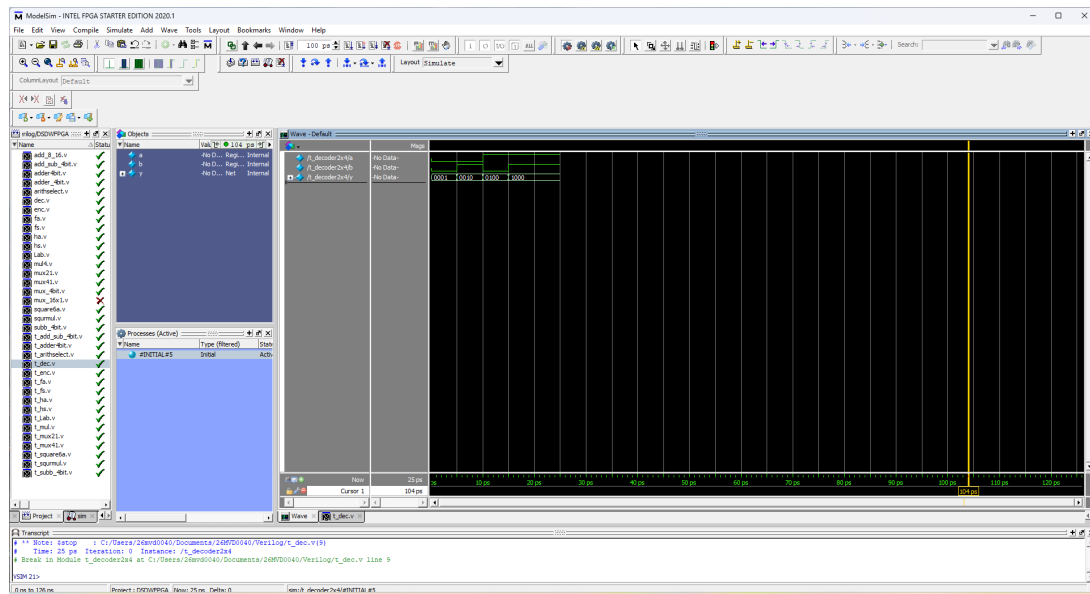


Figure 5: Simulation Output

6 Program 6 : 4-to-2 Encoder

6.1 Verilog Code

```
1
2 module enc(Y, D);
3   output [1:0] Y;
4   input [3:0] D;
5   or (Y[1], D[2], D[3]);
6   or (Y[0], D[1], D[3]);
7 endmodule
```

6.2 Testbench

```
1
2 module t_enc();
3   reg [3:0] d;
4   wire [1:0] y;
5   enc e1(y, d);
6   initial
7   begin
8     d = 4'b0001;
9     #5 d = 4'b0010;
10    #5 d = 4'b0100;
11    #5 d = 4'b1000;
12    #10 $stop;
13  end
14 endmodule
```

6.3 Simulation Output

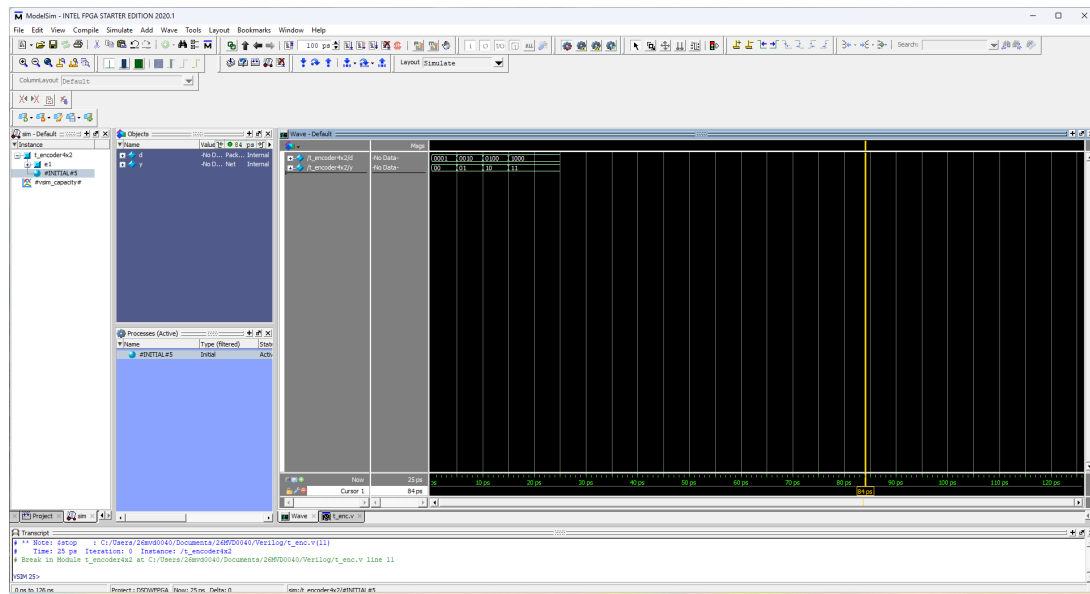


Figure 6: Simulation Output

7 Program 7 : 2-to-1 Multiplexer

7.1 Verilog Code

```
1 module mux21(Y, a, b, s);
2 output Y;
3 input a, b, s;
4 wire w1, w2, w3;
5 not (w1, s);
6 and (w2, a, w1);
7 and (w3, b, s);
8 or (Y, w2, w3);
9 endmodule
10
```

7.2 Testbench

```
1 module t_mux2x1();
2 reg a, b, s;
3 wire y;
4 mux21 m1(y, a, b, s);
5 initial
6 begin
7 a = 0; b = 1; s = 0;
8 #5 s = 1;
9 #5 a = 1;
10 b = 0; s = 0;
11 #5 s = 1;
12 #10 $stop;
13 end
14 endmodule
15
```

7.3 Simulation Output

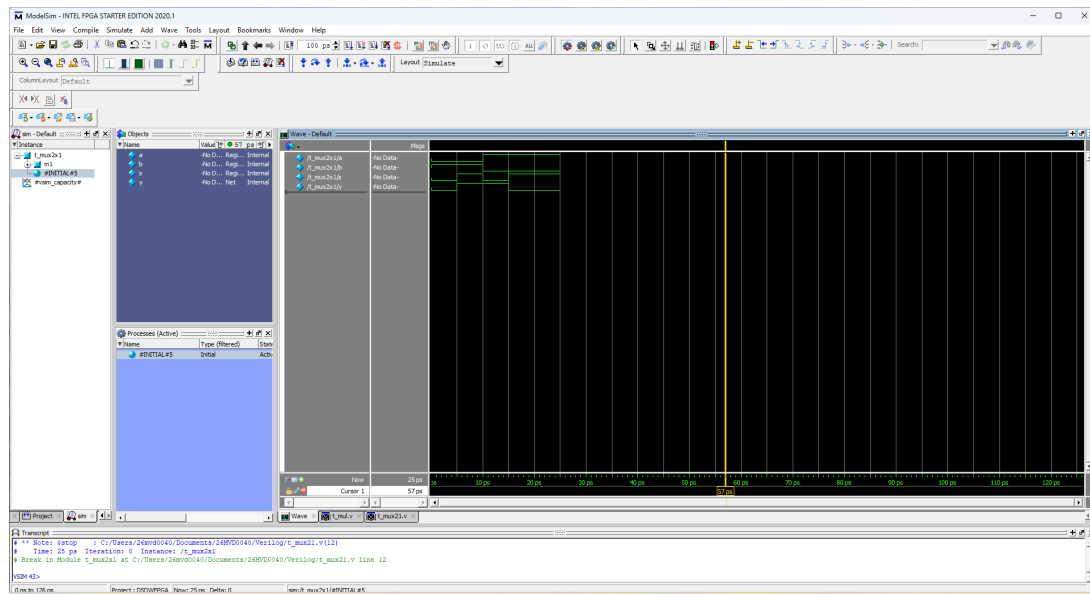


Figure 7: Simulation Output

8 Program 8 : 4-to-1 Multiplexer

8.1 Verilog Code

```
1
2 module mux41(Y, i0, i1, i2, i3, s0, s1);
3   output Y;
4   input i0, i1, i2, i3;
5   input s0, s1;
6   wire w1, w2;
7   wire t0, t1, t2, t3;
8   not (w1, s0);
9   not (w2, s1);
10  and (t0, i0, w2, w1);
11  and (t1, i1, w2, s0);
12  and (t2, i2, s1, w1);
13  and (t3, i3, s1, s0);
14  or (Y, t0, t1, t2, t3);
15 endmodule
```

8.2 Testbench

```
1
2 module t_mux41();
3   reg i0, i1, i2, i3;
4   reg s0, s1;
5   wire y;
6   mux41 m1(y, i0, i1, i2, i3, s0, s1);
7   initial
8   begin
9     i0 = 0;
10    i1 = 1;
11    i2 = 0;
12    i3 = 1;
13    s1 = 0;
14    s0 = 0;
15    #5 s1 = 0; s0 = 1;
16    #5 s1 = 1; s0 = 0;
17    #5 s1 = 1; s0 = 1;
18    #10 $stop;
19  end
20 endmodule
```

8.3 Simulation Output

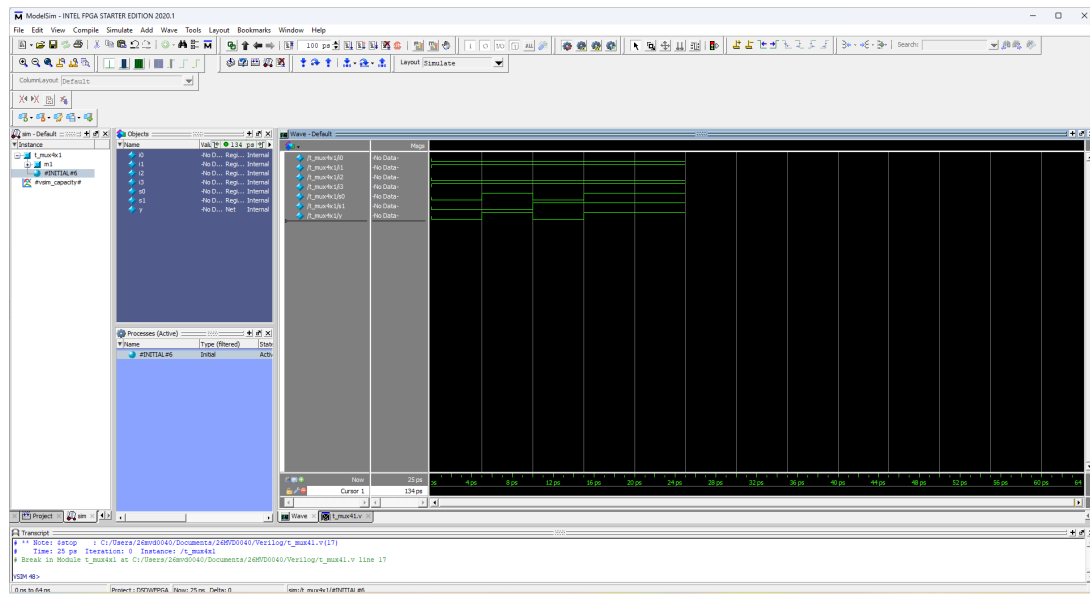


Figure 8: Simulation Output

9 Program 9 : 4-bit Ripple Carry Adder

9.1 Verilog Code

```
1 module adder4bit(Sum, Cout, A, B, Cin);
2 output [3:0] Sum;
3 output Cout;
4 input [3:0] A, B;
5 input Cin;
6 wire c1, c2, c3;
7 fa f1(Sum[0], c1, A[0], B[0], Cin);
8 fa f2(Sum[1], c2, A[1], B[1], c1);
9 fa f3(Sum[2], c3, A[2], B[2], c2);
10 fa f4(Sum[3], Cout, A[3], B[3], c3);
11 endmodule
```

9.2 Testbench

```
1 module t_adder4bit();
2 reg [3:0] a, b;
3 reg cin;
4 wire [3:0] sum;
5 wire cout;
6 adder4bit r1(sum, cout, a, b, cin);
7 initial
8 begin
9 a = 4'b0001; b = 4'b0010; cin = 0;
10 #5 a = 4'b0100; b = 4'b0011; cin = 0;
11 #5 a = 4'b1111; b = 4'b0001; cin = 0;
12 #10 $stop;
13 end
14 endmodule
```

9.3 Simulation Output

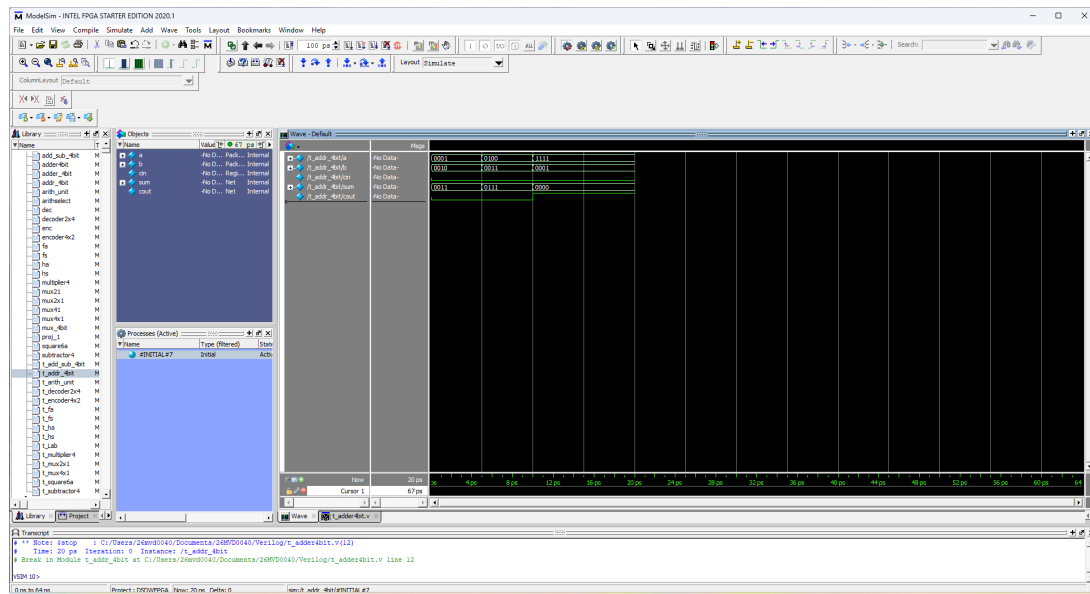


Figure 9: Simulation Output

10 Program 10 : 4-bit Adder/Subtractor

10.1 Verilog Code

```
1
2 module add_sub_4bit(Y, Cout, A, B, s);
3
4 output [3:0] Y;
5 output Cout;
6
7 input [3:0] A, B;
8 input s;
9
10 wire [3:0] sum;
11 wire [3:0] diff;
12 wire c1;
13
14 adder4bit A1(sum, Cout, A, B, 1'b0);
15
16 wire [3:0] B_comp;
17 assign B_comp = ~B;
18
19 adder4bit A2(diff, c1, A, B_comp, 1'b1);
20
21 mux21 M0(Y[0], sum[0], diff[0], s);
22 mux21 M1(Y[1], sum[1], diff[1], s);
23 mux21 M2(Y[2], sum[2], diff[2], s);
24 mux21 M3(Y[3], sum[3], diff[3], s);
25
26 endmodule
```

10.2 Testbench

```
1
2 module t_add_sub_4bit();
3
4 reg [3:0] A, B;
5 reg s;
6 wire [3:0] Y;
7 wire Cout;
8
9 add_sub_4bit U1(Y, Cout, A, B, s);
10
11 initial
12 begin
13     A = 4'b0101; B = 4'b0011; s = 0; // 5+3=8
14     #5;
15
16     A = 4'b0101; B = 4'b0011; s = 1; // 5-3=2
```

```

17     #5;
18
19     A = 4'b1001; B = 4'b0100; s = 0; // 9+4=13
20     #5;
21
22     A = 4'b1001; B = 4'b0100; s = 1; // 9-4=5
23     #10 $stop;
24 end
25
26 endmodule

```

10.3 Simulation Output

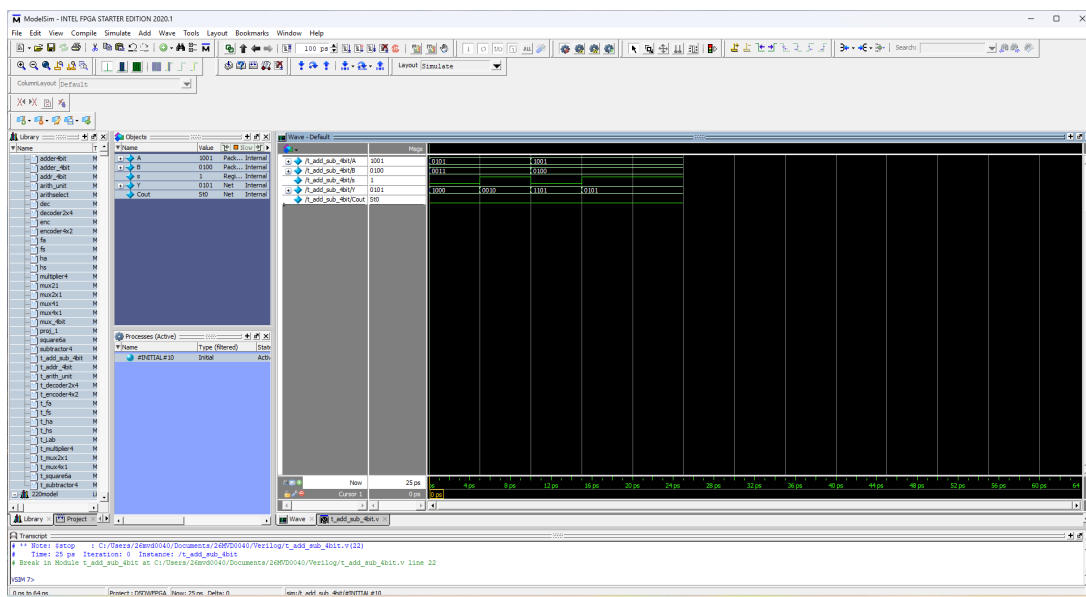


Figure 10: Simulation Output

11 Program 11 : 4-bit Multiplier

11.1 Verilog Code

```
1 module multiplier4(Y, A, B);
2 output [7:0] Y;
3 input [3:0] A, B;
4 wire p00, p01, p02, p03;
5 wire p10, p11, p12, p13;
6 wire p20, p21, p22, p23;
7 wire p30, p31, p32, p33;
8 wire [3:0] sum1, sum2, sum3;
9 wire c1, c2, c3;
10 and (p00, A[0], B[0]);
11 and (p01, A[1], B[0]);
12 and (p02, A[2], B[0]);
13 and (p03, A[3], B[0]);
14 and (p10, A[0], B[1]);
15 and (p11, A[1], B[1]);
16 and (p12, A[2], B[1]);
17 and (p13, A[3], B[1]);
18 and (p20, A[0], B[2]);
19 and (p21, A[1], B[2]);
20 and (p22, A[2], B[2]);
21 and (p23, A[3], B[2]);
22 and (p30, A[0], B[3]);
23 and (p31, A[1], B[3]);
24 and (p32, A[2], B[3]);
25 and (p33, A[3], B[3]);
26 buf (Y[0], p00);
27 adder4bit add1(sum1, c1, {1'b0, p03, p02, p01}, {p13, p12, p11,
28   p10}, 1'b0);
29 buf (Y[1], sum1[0]);
30 adder4bit add2(sum2, c2, {c1, sum1[3:1]}, {p23, p22, p21, p20},
31   1'b0);
32 buf (Y[2], sum2[0]);
33 adder4bit add3(sum3, c3, {c2, sum2[3:1]}, {p33, p32, p31, p30},
34   1'b0);
35 buf (Y[3], sum3[0]);
36 buf (Y[4], sum3[1]);
37 buf (Y[5], sum3[2]);
38 buf (Y[6], sum3[3]);
39 buf (Y[7], c3);
40 endmodule
```

11.2 Testbench

1

```

2 module t_multiplier4();
3 reg [3:0] A, B;
4 wire [7:0] Y;
5 multiplier4 m1(Y, A, B);
6 initial
7 begin
8   A = 4'd2; B = 4'd3;
9   #5 A = 4'd4; B = 4'd5;
10  #5 A = 4'd7; B = 4'd2;
11  #10 $stop;
12 end
13 endmodule

```

11.3 Simulation Output

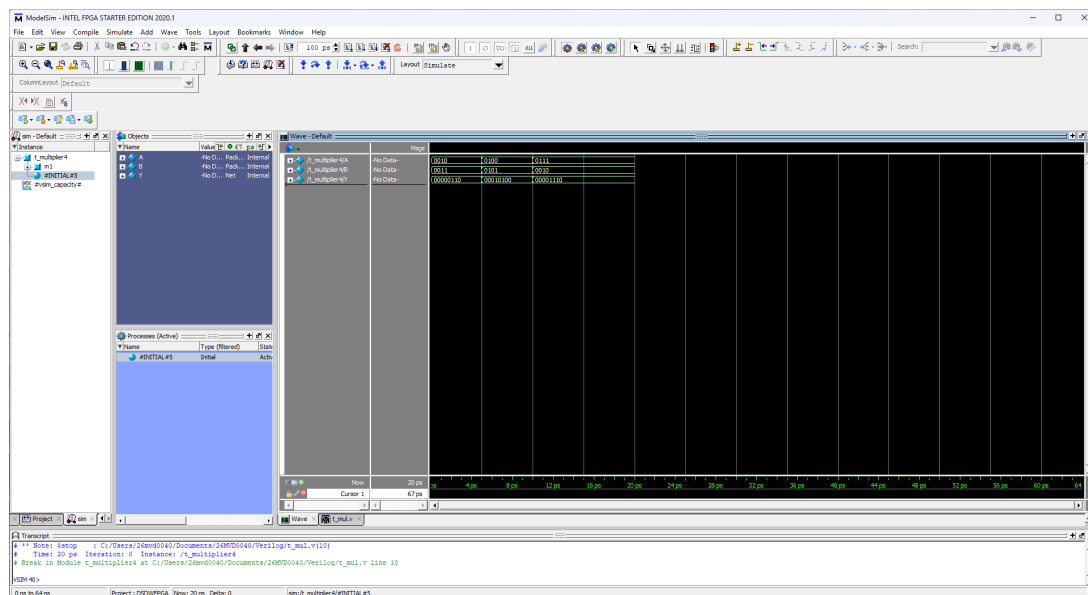


Figure 11: Simulation Output

12 Program 12 : Arithmetic Unit (S=0: Y=A-B, S=1: Y=B-A)

12.1 Verilog Code

```
1
2 module arithselect(Y, A, B, S);
3
4 output [3:0] Y;
5 input [3:0] A, B;
6 input S;
7
8 wire [3:0] diff_AB;
9 wire [3:0] diff_BA;
10 wire c1, c2;
11
12 adder4bit sub1(diff_AB, c1, A, ~B, 1'b1);
13 adder4bit sub2(diff_BA, c2, B, ~A, 1'b1);
14
15 mux21 m0(Y[0], diff_AB[0], diff_BA[0], S);
16 mux21 m1(Y[1], diff_AB[1], diff_BA[1], S);
17 mux21 m2(Y[2], diff_AB[2], diff_BA[2], S);
18 mux21 m3(Y[3], diff_AB[3], diff_BA[3], S);
19
20 endmodule
```

12.2 Testbench

```
1
2 module t_arith_unit();
3 reg [3:0] a, b;
4 reg s;
5 wire [3:0] y;
6 arith_unit a1(y, a, b, s);
7 initial
8 begin
9 a = 4'b1100; b = 4'b1001; s = 0;
10 #5 a = 4'b1100; b = 4'b1001; s = 1;
11 #10 $stop;
12 end
13 endmodule
```

12.3 Simulation Output

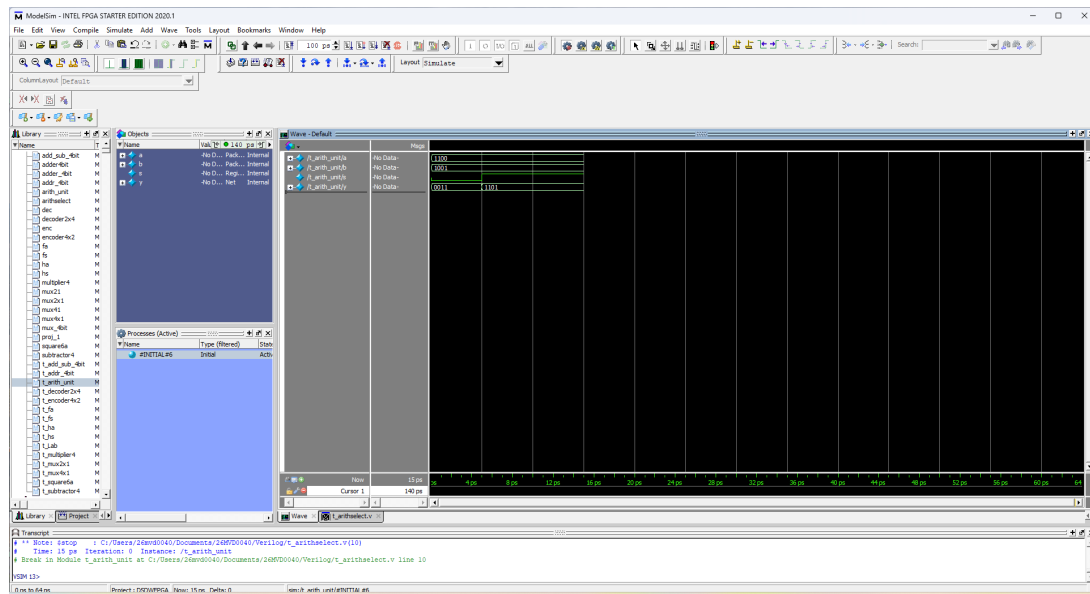


Figure 12: Simulation Output

13 Program 13 : Arithmetic Circuit ($Y=A^2 + 6A$)

13.1 Verilog Code

```
1 module square6a(Y, A);
2 output [7:0] Y;
3 input [3:0] A;
4 wire [7:0] a_squared;
5 wire [7:0] six_a;
6 wire c_out;
7 multiplier4 mult1(a_squared, A, A);
8 multiplier4 mult2(six_a, 4'd6, A);
9 adder4bit add_low (Y[3:0], c_out, a_squared[3:0], six_a[3:0], 1'b0);
10 adder4bit add_high(Y[7:4], , a_squared[7:4], six_a[7:4], c_out);
11 endmodule
12
```

13.2 Testbench

```
1 module t_square6a();
2 reg [3:0] a;
3 wire [7:0] y;
4 square6a s1(y, a);
5 initial
6 begin
7 a = 4'b1111;
8 #5 a = 4'b1000;
9 #10 $stop;
10 end
11 endmodule
12
```

13.3 Simulation Output

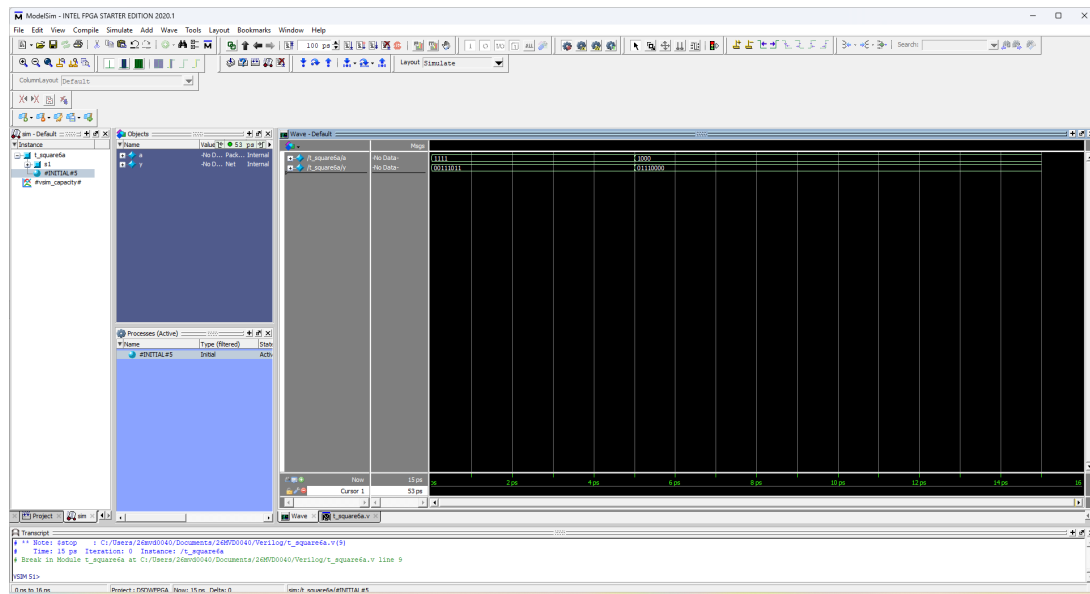


Figure 13: Simulation Output

14 Program 14 : FPGA Design Implementation Experiment

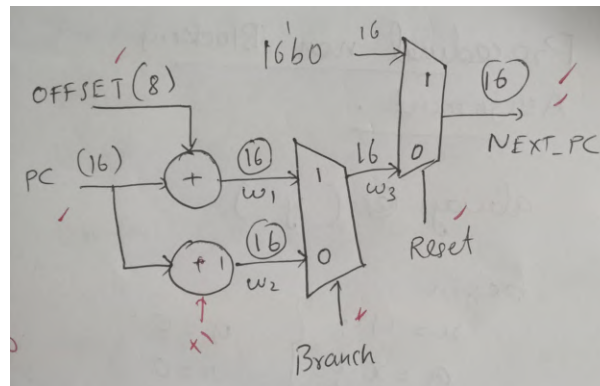


Figure 14: Given Design

14.1 Verilog Code

```
1 module addr_16bit(w1,offset,pc);
2 output [0:15] w1;
3 input [0:7] offset;
4 input [0:15] pc;
5 assign w1= offset + pc;
6
7
8 endmodule
```

14.2 Verilog Code

```
1 module mux_16x1(w1,w2,w3,branch);
2 output [0:15] w3;
3 input [0:15] w1,w2;
4 input branch;
5
6
7 assign w3 = branch ? w1 : w2;
8
9 endmodule
```

14.3 Verilog Code

```
1 module Lab(nextpc,reset,branch,pc);
2
3
4 output [15:0] nextpc;
5 input [15:0] pc;
```

```

6 input reset,branch;
7
8 wire [7:0]offset;
9 wire [15:0]w1,w2,w3;
10
11 assign offset= 8'd5;
12
13 addr_16bit a1(w1,offset,pc);
14 addr_16bit a2(w2,16'd1,pc);
15
16 mux_16x1 m1(w1,w2,w3,branch);
17 mux_16x1 m2(16'd0,w3,nextpc,reset);
18 endmodule

```

14.4 Testbench

```

1
2
3
4 module t_Lab();
5
6 reg [15:0]pc;
7 reg reset,branch;
8
9 wire [15:0]nextpc;
10
11 Lab L1(nextpc,reset,branch,pc);
12
13 initial
14 begin
15     pc=16'd10 ;
16     reset = 0;
17     branch = 0;
18
19     #5 branch = 0; reset = 0 ;
20     #5 branch = 0; reset = 1 ;
21     #5 branch = 1; reset = 0 ;
22     #5 branch = 1; reset = 1 ;
23     #10 $stop;
24 end
25 endmodule

```

14.5 Pin Assignment

```

1
2 set_location_assignment PIN_G19 -to nextpc[0]
3 set_location_assignment PIN_F19 -to nextpc[1]
4 set_location_assignment PIN_E19 -to nextpc[2]
5 set_location_assignment PIN_F21 -to nextpc[3]

```

```

6  set_location_assignment PIN_F18 -to nextpc[4]
7  set_location_assignment PIN_E18 -to nextpc[5]
8  set_location_assignment PIN_J19 -to nextpc[6]
9  set_location_assignment PIN_H19 -to nextpc[7]
10 set_location_assignment PIN_J17 -to nextpc[8]
11 set_location_assignment PIN_G17 -to nextpc[9]
12 set_location_assignment PIN_J15 -to nextpc[10]
13 set_location_assignment PIN_H16 -to nextpc[11]
14 set_location_assignment PIN_J16 -to nextpc[12]
15 set_location_assignment PIN_H17 -to nextpc[13]
16 set_location_assignment PIN_F15 -to nextpc[14]
17 set_location_assignment PIN_G15 -to nextpc[15]
18
19 set_location_assignment PIN_AB28 -to pc[0]
20 set_location_assignment PIN_AC28 -to pc[1]
21 set_location_assignment PIN_AC27 -to pc[2]
22 set_location_assignment PIN_AD27 -to pc[3]
23 set_location_assignment PIN_AB27 -to pc[4]
24 set_location_assignment PIN_AC26 -to pc[5]
25 set_location_assignment PIN_AD26 -to pc[6]
26 set_location_assignment PIN_AB26 -to pc[7]
27 set_location_assignment PIN_AC25 -to pc[8]
28 set_location_assignment PIN_AB25 -to pc[9]
29 set_location_assignment PIN_AC24 -to pc[10]
30 set_location_assignment PIN_AB24 -to pc[11]
31 set_location_assignment PIN_AB23 -to pc[12]
32 set_location_assignment PIN_AA24 -to pc[13]
33 set_location_assignment PIN_AA23 -to pc[14]
34 set_location_assignment PIN_AA22 -to pc[15]
35 set_location_assignment PIN_Y24 -to reset
36 set_location_assignment PIN_Y23 -to branch

```

14.6 Simulation Output

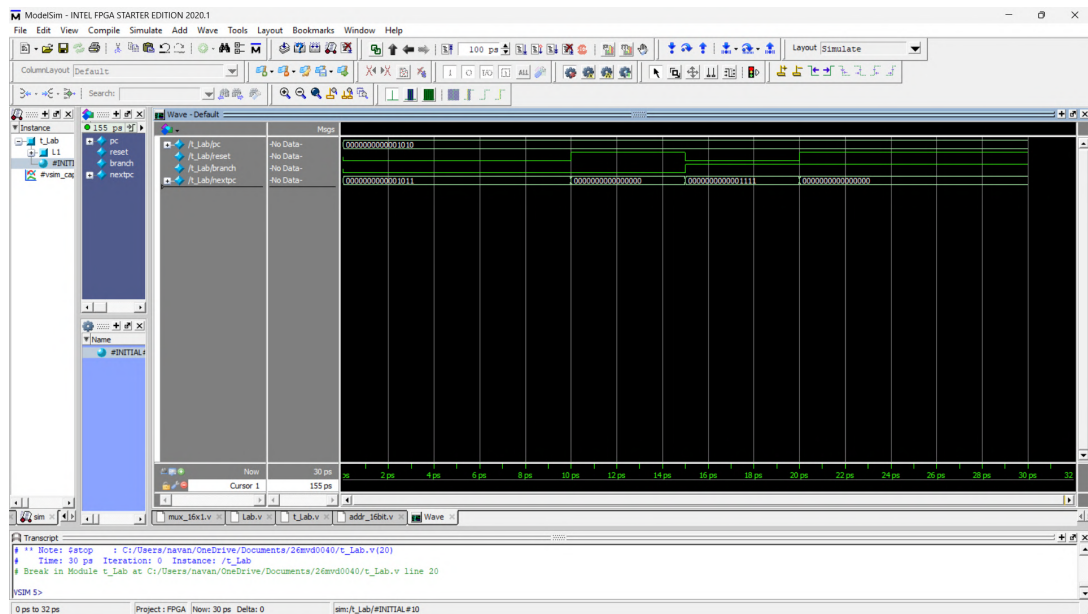


Figure 15: Simulation Output

14.7 FPGA Implementation Output

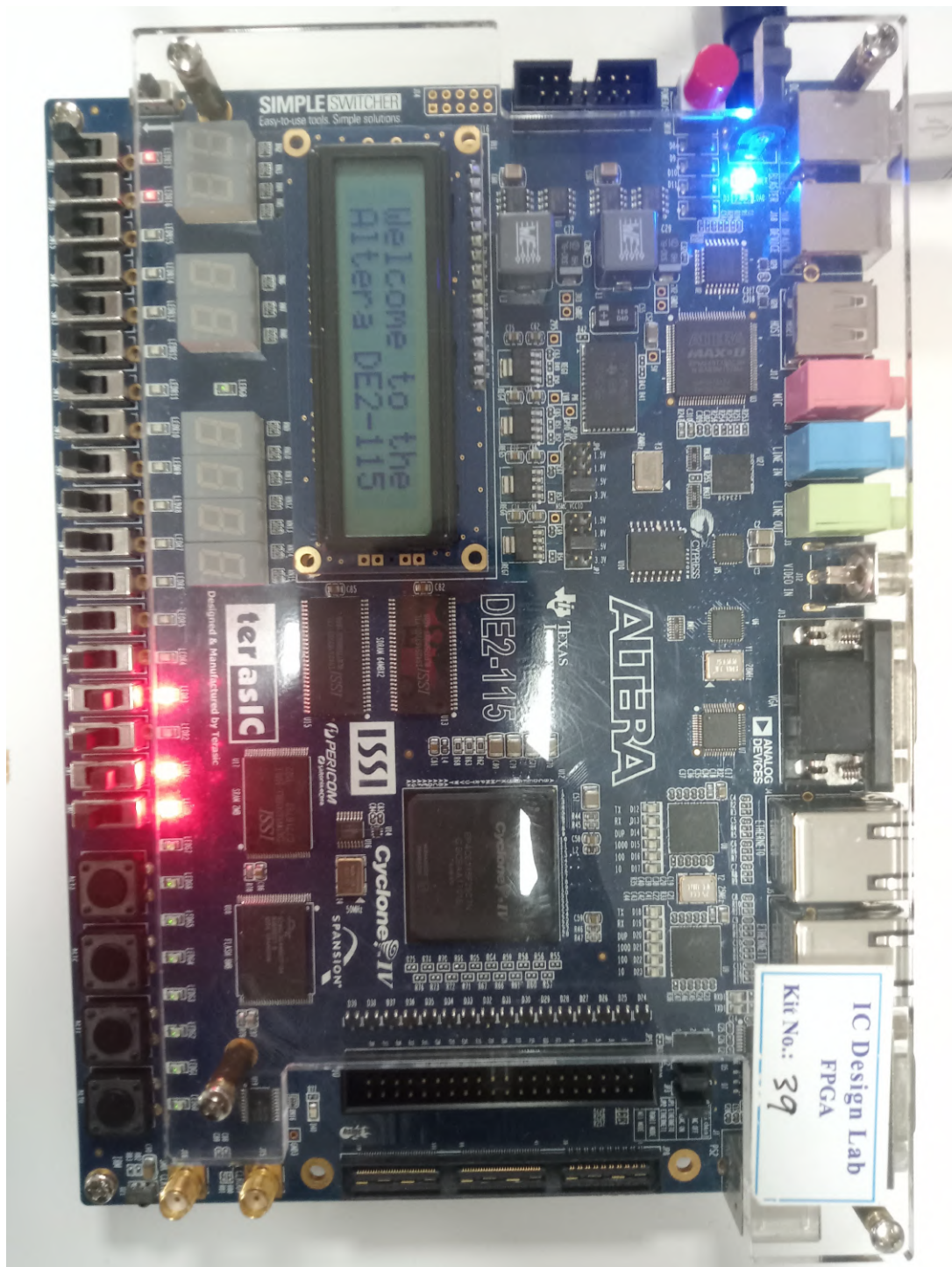


Figure 16: FPGA Implementation Output $B = 0$ $R = 0$

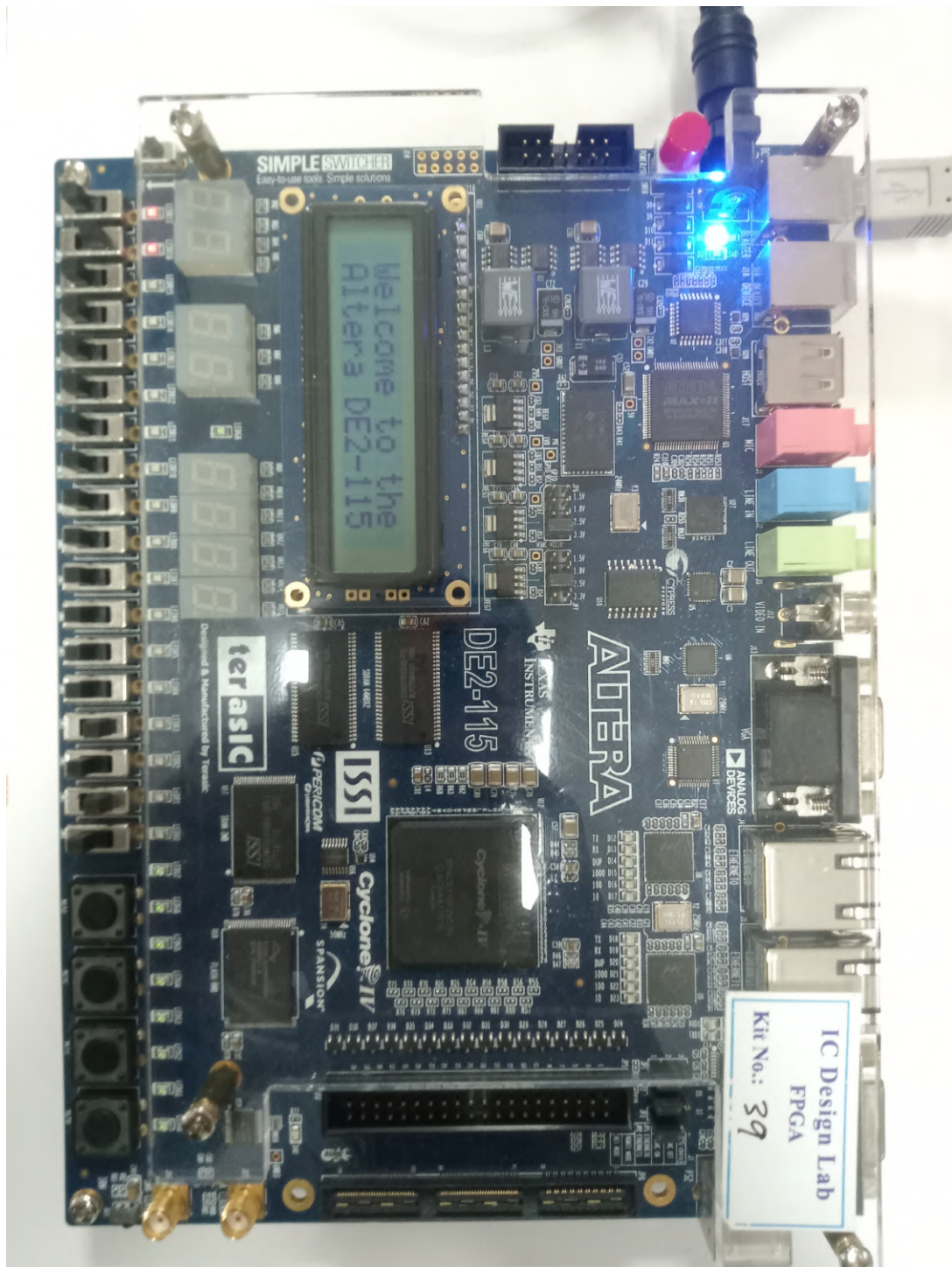


Figure 17: FPGA Implementation Output $B = 0$ $R = 1$

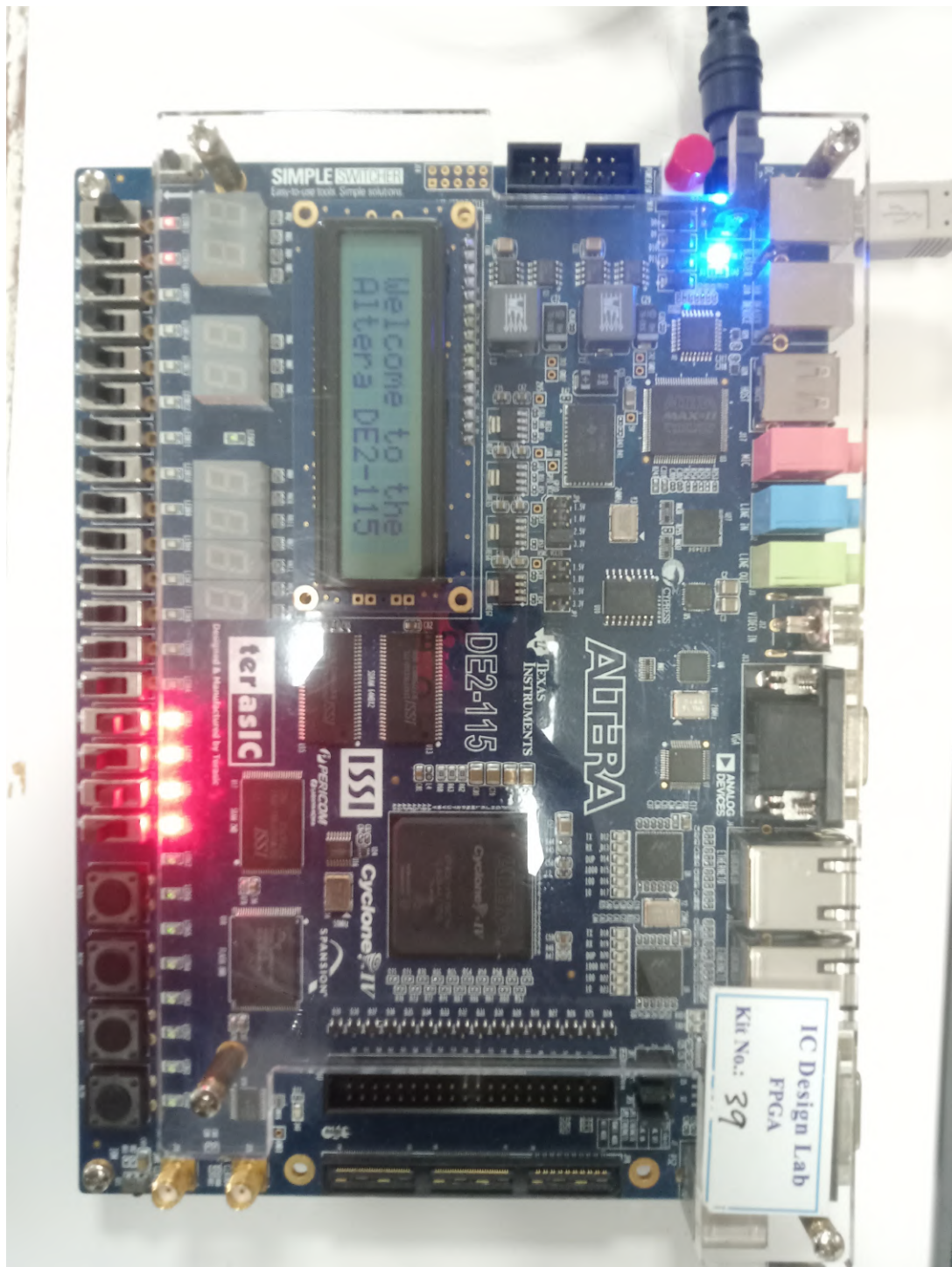


Figure 18: FPGA Implementation Output $B = 1$ $R = 0$

14.8 FPGA Implementation Output

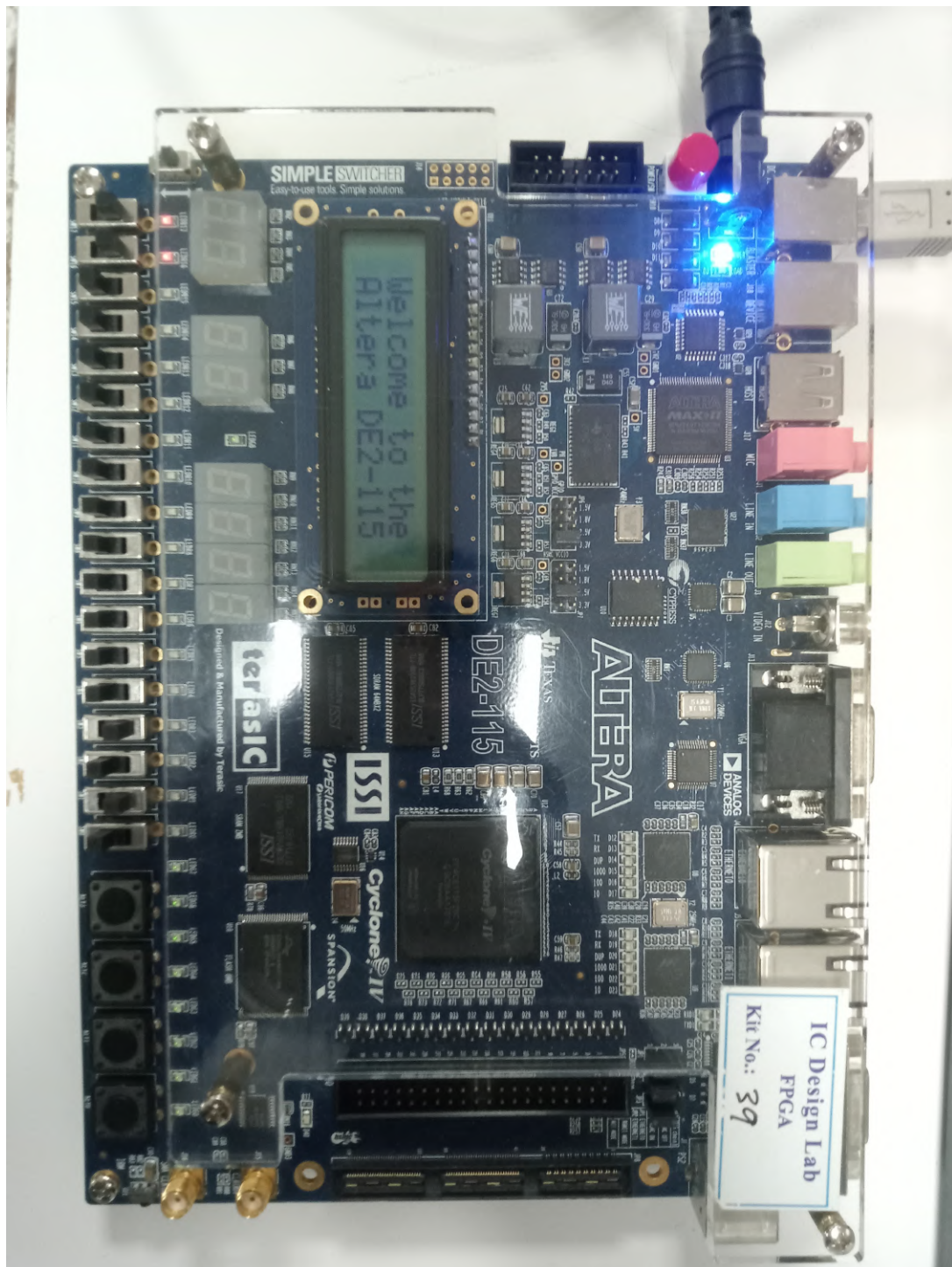


Figure 19: FPGA Implementation Output $B = 1$ $R = 1$